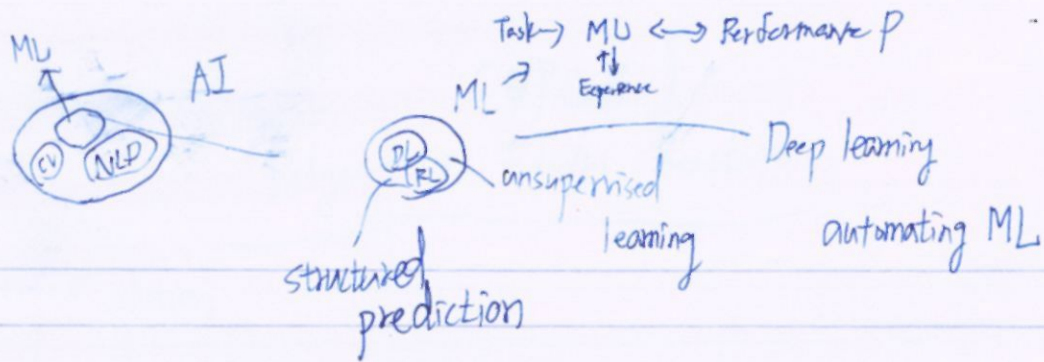




Facial recognition

Recommendation system

before

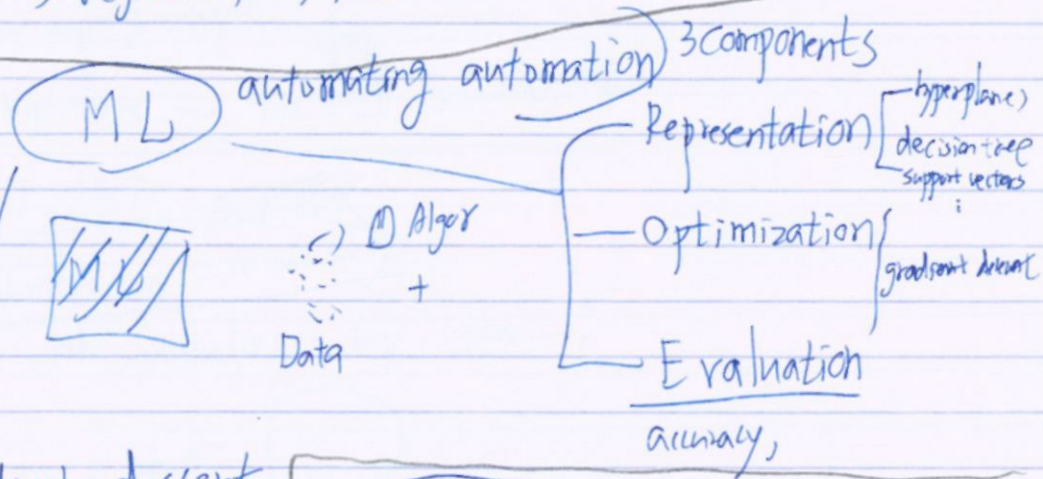
rule-based translation (~2000) $\rightarrow$ ML

Medical diagnosis

k-nearest Neighbors $\begin{cases} \text{classification} \\ \text{regression} \end{cases}$

ML basics + linear algebra review
 HW: income classification

linear classification $\hookrightarrow$ regression $\rightarrow$ DL



gradient descent

learning rate

— partially observed learning:

learning setting

supervised learning $f(x)=y$ $\begin{cases} \text{class: input: picture, out: type} \\ \text{regression: input: test, out: value} \end{cases}$

unsupervised learning: 分群, 降维 only (keep important structure)

semi-supervised learning

Output y : regression
e.g. price
or
classification

or
structured
e.g. semantic segmentation

Supervised Learning

$$D = \{(x_i, y_i)\}_{i=1}^n$$

scenario:
① changing frequently

$$f(x_i) = y_i$$

classification

binary
multiclass
structured

1. set up a supervised learning problem
2. Data collection (x, y)
3. Representation $x \rightarrow$ proper form
4. Modeling: hypotheses space
5. Learning
6. Model Selection
7. go or repeat

Goal: min loss function
by e.g. gradient descent

GD: $\begin{cases} \text{Batch} \\ \text{Stochastic} \\ \text{min-batch} \end{cases}$

If loss function is convex, gd is guaranteed to converge to the global min

target: $y = w^T x \Rightarrow y = w^T x + \epsilon$ ^{noise}

Linear regression

$\begin{cases} \text{MSE} \\ \text{MAE} \\ \text{Huber loss} \end{cases}$

How to define different M ?

Max likelihood estimation

Data log likelihood:

$$\sum \log p(x_i | w)$$

sigmoid
↑

$$\phi l(w) = \sum [y_i \log \sigma(w^T x_i) + (1 - y_i) \log (1 - \sigma(w^T x_i))]$$

(Gradient) $\partial l(w) = \sum \left[\frac{y_i}{\sigma(w^T x_i)} \sigma'(w^T x_i) - \frac{(1 - y_i)}{1 - \sigma(w^T x_i)} \sigma'(w^T x_i) \right]$

$$= \sum (y_i - \sigma(w^T x_i)) x_i$$

target: data $X \rightarrow$ label y

Model $f(x) = \sigma(z)$

w and x



Hypothesis function $\rightarrow \sigma(w^T x)$



$[0, 1]$



[MLE, MAP]

① make a prediction

if error obeys Gaussian Distribution

we can use Mean Squared error

② Evaluation

[Here is Objective function]

way A: log-likelihood Function

max ~~target~~ ^{it} function $J(w)$

way B: Loss Function $-J(w) = \text{Loss}(w)$

↑
min it

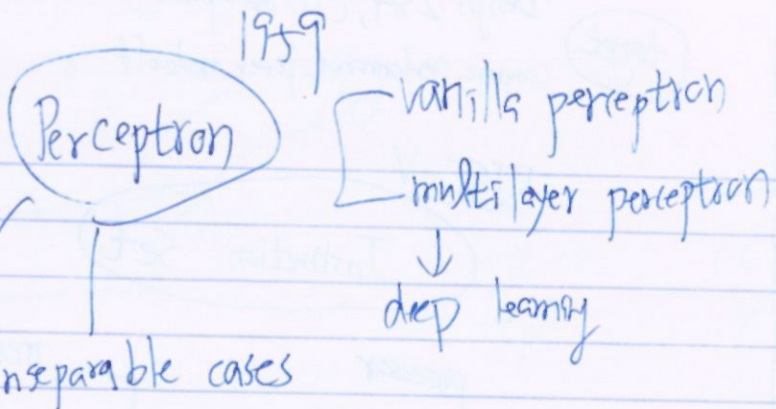
③ find direction

$\Delta \text{Loss}(w)$

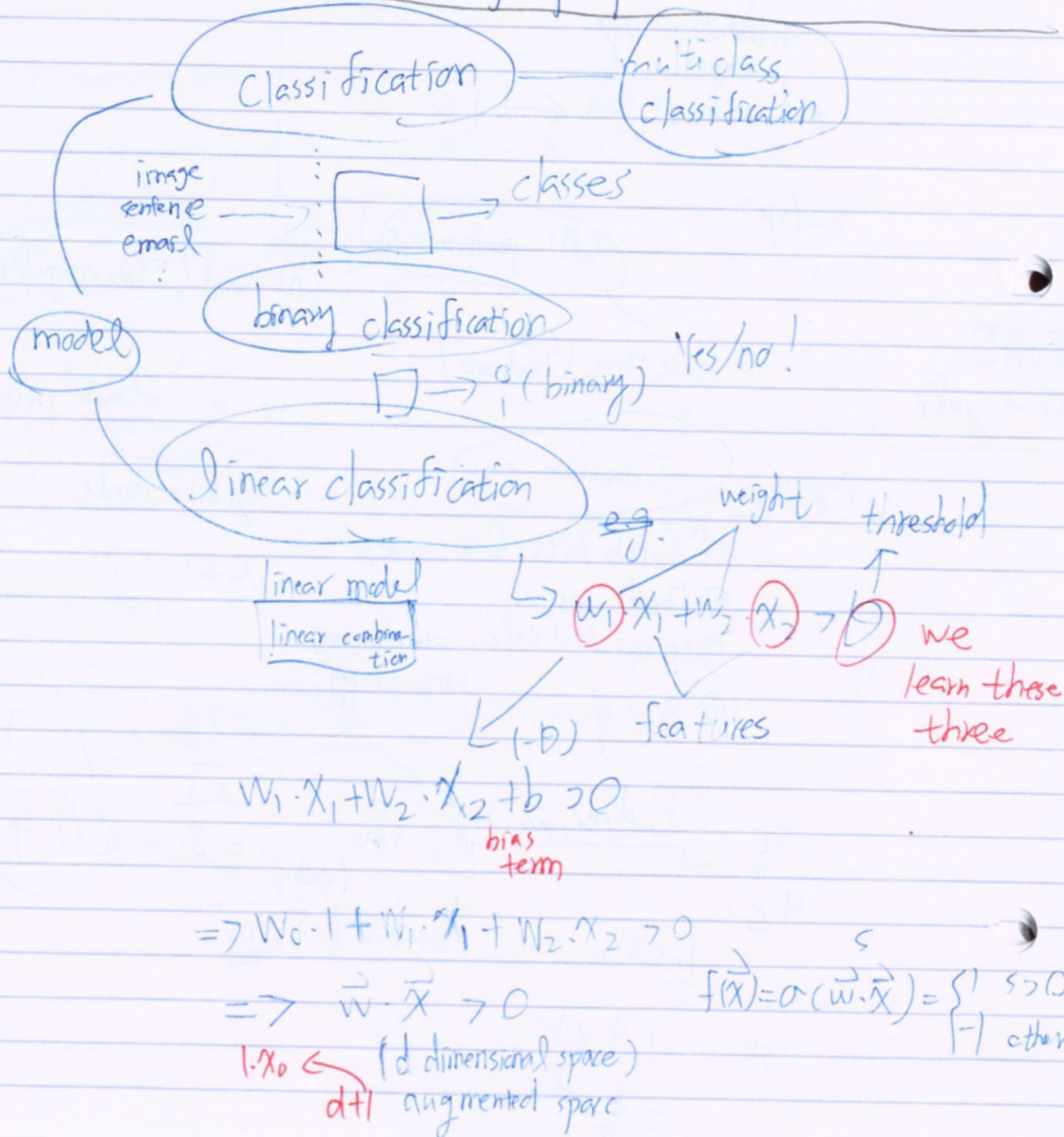
传播到 all w_i
controller

④ Update model

$w_{new} = w_{old} - \alpha \Delta \text{Loss } w_{old}$
(Gradient Descent)



large-margin classifiers (1999)
voted and averaged perceptron



Perceptron Algor

1. $\vec{w} \leftarrow (0, 0, 0 \dots 0) \Rightarrow \text{output: } \checkmark$

2. find error $y \cdot (\vec{w} \cdot \vec{x}) \leq 0$

3. update $\vec{w} \Rightarrow \vec{w}' \leftarrow \vec{w} + \vec{x}$ (for positive case)

$$\vec{w}' \cdot \vec{x} = \vec{w} \cdot \vec{x} + \underbrace{\vec{x} \cdot \vec{x}}_{> 0} > \vec{w} \cdot \vec{x} \quad \vec{w}' \leftarrow \vec{w} - \vec{x} \text{ (neg)} \\ \Rightarrow \vec{w}' \leftarrow \vec{w} + y \cdot \vec{x}$$

4. Convergence $\rightarrow$ (best case) in a round, all examples are classified correctly by the $\vec{w}$

$\Rightarrow$ (General) $\circ$ max limit iteration

$\oplus$ dev set decide the best iteration

Linear separation $\Rightarrow$ Perceptron convergence theorem

support

$\rightarrow$ SVM

find the line

that $\max \geq \delta$ (linear buffer zone)

D has δ (linear separable)

$\Rightarrow$ perceptron algorithm must CONVERGE before $\frac{R^2}{\delta^2}$

$$\text{mistakes (Updates)} \leq \frac{R^2}{\delta^2} \quad R = \max_{(\vec{x}, y) \in D} \|\vec{x}\| \quad (\text{radius})$$

Handle Inseparable case

original feature space $\xrightarrow{\text{project}}$ higher-dimensional feature space

$\Rightarrow$ (possible) linear separable

$\leftarrow$ Kernel trick of SVM ... methods

not linearly separable

XOR example

x_1	x_2	y
+	+	+
+	-	-
-	-	+
-	+	-

$y = x_1 \times x_2 \Rightarrow$
nonlinear relation

odd feature transformation
kernel method

separated by the plane $y=0$ → project back
got nonlinear decision boundary in low dimensions

大, 小
 $\Phi = \phi = f \circ \phi$
fuge

In general,
 $\Rightarrow$ linearly separable + feature map

$\Phi(\cdot)$: feature map $y(\vec{u} \cdot \Phi(\vec{x})) \geq \delta$, for all $(\vec{x}, y) \in D$

epoch:
完整训练一次

oracle vector
 $\|\vec{u}\|=1$

mistakes (updates) $\leq \frac{R_{\Phi}^2(D)}{\delta_{\Phi}^2(D)}$

improvement of
vanilla perceptron

★ voted perceptron ① much less sensitive
averaged perceptron ② large margin

Prediction = $\text{sign} \left(\sum_{k=1}^M c_k \times \text{sign}(u_k \cdot x_{\text{test}}) \right)$

Regression

target: $\min[\text{diff}(\text{prediction}, \text{truth})]$

measured by $\rightarrow$ mean squared error (MSE)

$$y = w \cdot s + b \Rightarrow y = \underbrace{w_1 \cdot s + w_2 \cdot r + b}_{\text{match target}}$$

$\downarrow$ finally
make predictions on new data

(features)
 $X = (x_0=1, x_1, x_2, \dots, x_d) \Rightarrow d+1$ dimensional augmented space

$$\vec{w} = (w_0, w_1, w_2, \dots, w_d)$$

$$\Rightarrow \hat{y} = \vec{w} \cdot \vec{x}, \hat{y} = \text{predicted value}$$

是 2 還是 n 沒影響

量化成果 $\Rightarrow$ cost function: $J(\vec{w}) = \frac{1}{2} \sum_{(\vec{x}, y) \in D} (f_{\vec{w}}(\vec{x}) - y)^2$
as small as possible

Regularization

$\lambda > 0$ (hyperparameter)

optimize (min)
iteratively

$$L_2: J(\vec{w}) = \frac{1}{2} \sum_{(\vec{x}, y) \in D} (f_{\vec{w}}(\vec{x}) - y)^2 + \lambda \|\vec{w}\|_2^2$$

aka. regularization strength

$$\lambda (\sqrt{w_0^2 + w_1^2 + w_2^2 + \dots + w_n^2})^2$$

optimize (min)

$$L_1: J(\vec{w}) = \frac{1}{2} \sum_{(\vec{x}, y) \in D} (f_{\vec{w}}(\vec{x}) - y)^2 + \lambda \|\vec{w}\|_1$$

$$\lambda (|w_0| + |w_1| + |w_2| + \dots)$$

Gradient Descent

optimize (min) cost function iteratively

or

LMS algor. $\vec{w} \leftarrow \vec{w} + \alpha (y - \vec{w} \cdot \vec{x}) \vec{x}$ (one example)

(least mean squares)

more data's

$$\Rightarrow \vec{w} \leftarrow \vec{w} + \alpha \sum_{(\vec{x}, y) \in D} (y - \vec{w} \cdot \vec{x}) \vec{x}$$

每次更新都是需要完整的数据 set

Normal Equation

for large-scale datasets

SGD

Stochastic Gradient Descent

randomly picked from D

BGD

$$\vec{w} \leftarrow \vec{w} + \alpha \sum_D (y - \vec{w} \cdot \vec{x}) \vec{x} \xrightarrow{\text{simplify}} \vec{w}' \leftarrow \vec{w} + \alpha (y^{(i)} - \vec{w} \cdot \vec{x}^{(i)}) \vec{x}^{(i)}$$

faster convergence but less precise

stability $\longleftrightarrow$ efficiency



Mini-Batch Gradient Descent

Normal Equation

another way to min $J(\vec{w})$